



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 8.1

“Cronómetro de 100 minutos”

Grupo: 6CM1

Integrantes

- **García Escamilla Bryan Alexis**
- **Meléndez Macedonio Rodrigo**

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 15 de abril de 2025

INTRODUCCIÓN

Cronómetro de 100 minutos

Estructura de la cascada de conteo

Para cubrir este rango, el sistema se organiza en cuatro etapas de contadores conectados en serie, donde cada una "avisa" a la siguiente cuando debe incrementar:

1. **Segundos (Unidades):** Contador MOD-10 (0-9). Se incrementa cada segundo exacto mediante la base de tiempo. Al pasar de 9 a 0, activa a las decenas de segundos.
2. **Segundos (Decenas):** Contador MOD-6 (0-5). Es crucial que sea MOD-6 para que el conteo de segundos se reinicie al llegar a 60. Al pasar de 5 a 0, activa a las unidades de minutos.
3. **Minutos (Unidades):** Contador MOD-10 (0-9). Se incrementa cada vez que los segundos completan un ciclo de 60.
4. **Minutos (Decenas):** Contador MOD-10 (0-9). Permite que el cronómetro llegue hasta el minuto 99.

La base del tiempo y precisión

En un rango de 100 minutos, el error acumulado puede ser muy notorio.

- Si se usa un oscilador interno (como el RC de un microcontrolador), el cronómetro podría desfasarse varios segundos al final de la hora.
- Por ello, se utilizan cristales de cuarzo externos (como el de 32.768 kHz o de varios MHz divididos mediante un prescaler), que garantizan que el pulso de "1 segundo" sea extremadamente estable frente a cambios de temperatura.

Visualización y multiplexación

Manejar cuatro displays de 7 segmentos de forma individual requeriría 28 pines (7 x 4). Para optimizar esto en electrónica, se utiliza la Multiplexación por División de Tiempo:

- Los 7 segmentos de todos los displays están conectados en paralelo al mismo bus de datos.
- Se activa el "común" del primer display y se envía el dato de las unidades de segundo.
- Se apaga el primero, se activa el segundo y se envía el dato de las decenas de segundo.
- Este ciclo se repite tan rápido que el ojo humano percibe los cuatro dígitos encendidos simultáneamente sin parpadeo.

Lógica de control y estados

Un cronómetro de este tipo suele operar bajo una Máquina de Estados (FSM) que gestiona los botones:

- **Start/Stop:** Habilita o deshabilita la señal de reloj que entra al primer contador.
- **Lap (Vuelta):** Congela la visualización del display pero permite que los contadores internos sigan corriendo (utilizando un registro intermedio o "Latch").
- **Clear/Reset:** Envía un pulso de limpieza a todos los flip-flops de la cascada simultáneamente.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 8.1, el circuito a armar es el siguiente:

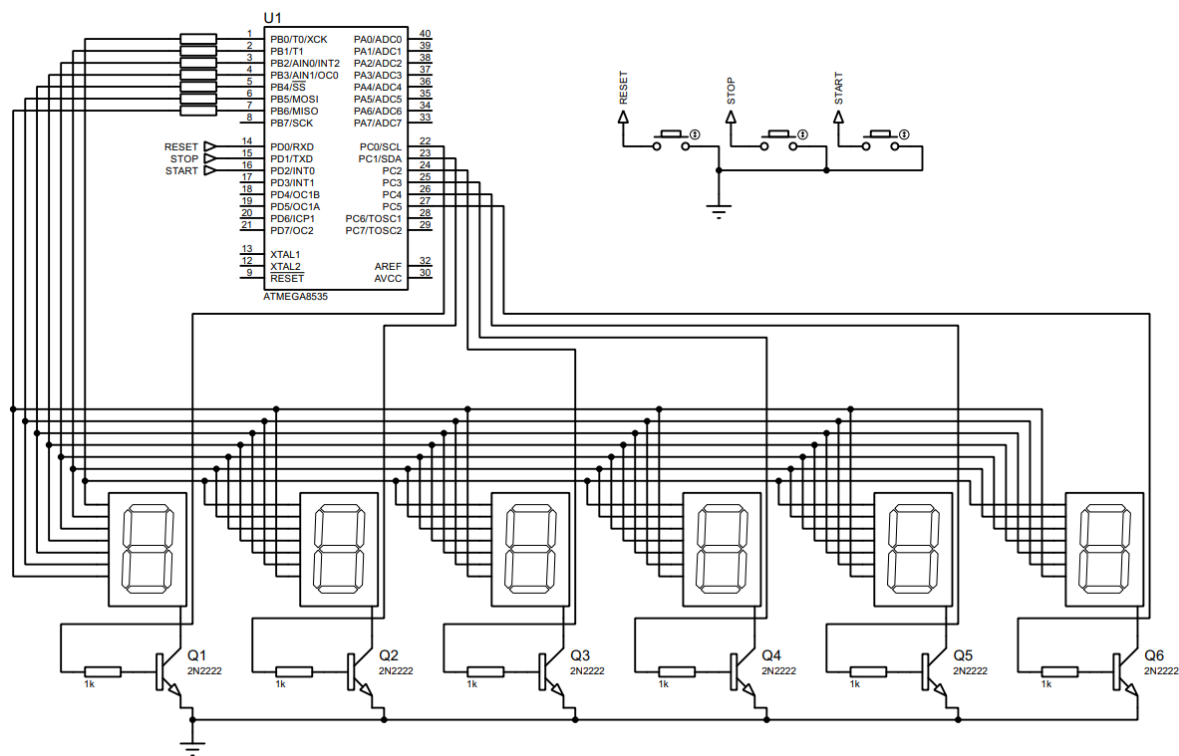
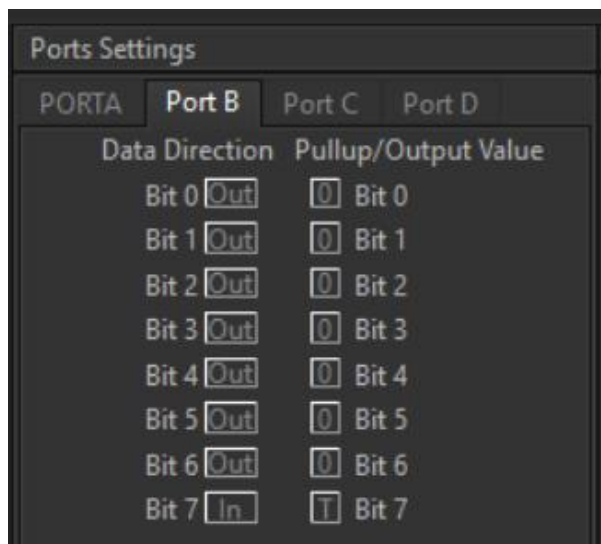


Imagen 1. Circuito creado en Proteus.

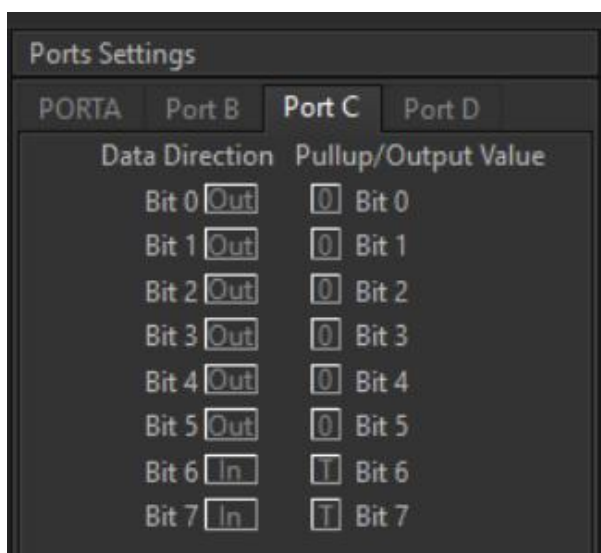
Configuración previa al código

De acuerdo con la imagen, el registro B se configura como de salida (PB0-PB6) al igual que el registro C (PC0-PC5), mientras que el registro D como de entrada (PD0-PD2) activando las resistencias de pull-up.



Ports Settings			
PORTA	Port B	Port C	Port D
Data Direction		Pullup/Output Value	
Bit 0	Out	0	Bit 0
Bit 1	Out	0	Bit 1
Bit 2	Out	0	Bit 2
Bit 3	Out	0	Bit 3
Bit 4	Out	0	Bit 4
Bit 5	Out	0	Bit 5
Bit 6	Out	0	Bit 6
Bit 7	In	T	Bit 7

Imagen 2. Configuración del puerto B.



Ports Settings			
PORTA	Port B	Port C	Port D
Data Direction		Pullup/Output Value	
Bit 0	Out	0	Bit 0
Bit 1	Out	0	Bit 1
Bit 2	Out	0	Bit 2
Bit 3	Out	0	Bit 3
Bit 4	Out	0	Bit 4
Bit 5	Out	0	Bit 5
Bit 6	In	T	Bit 6
Bit 7	In	T	Bit 7

Imagen 3. Configuración del puerto C.

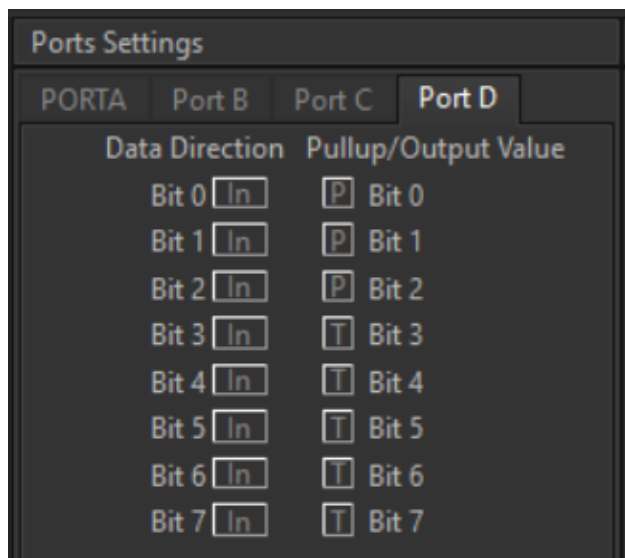


Imagen 4. Configuraci3n del puerto D.

C3digo del circuito de CodeVision:

```

1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4  Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr3ctica 8 'Cron3metro de 100 minutos'
8  Version : 1.0
9  Date    : 31/03/2026
10 Author  : Garc3a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type      : ATmega8535
16 Program type   : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model   : Small
19 External RAM size : 0
20 Data Stack size : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #include <delay.h>
26
27 #define RESET_BUTTON (PIND.0 == 0)
28 #define STOP_BUTTON  (PIND.1 == 0)
29 #define START_BUTTON (PIND.2 == 0)

```

```

31 // Declare your global variables here
32 const unsigned char tabla_7_segmentos[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
33
34 volatile unsigned char minutos_decenas = 0;
35 volatile unsigned char minutos_unidades = 0;
36 volatile unsigned char segundos_decenas = 0;
37 volatile unsigned char segundos_unidades = 0;
38 volatile unsigned char centesimas_decenas = 0;
39 volatile unsigned char centesimas_unidades = 0;
40
41 volatile unsigned char display_actual = 0;
42 volatile unsigned char running = 0;
43 unsigned char i = 0;
44
45 void actualizar_displays(void);
46 void incrementar_tiempo(void);
47 void reset_tiempo(void);
48
49 #interrupt [TIM0_OVF] void timer0_ovf_isr(void) {
50     TCNT0 = 131; // ~1 ms con 1MHz y prescaler 8
51     actualizar_displays();
52 }
53
54 #interrupt [TIM1_OVF] void timer1_ovf_isr(void) {
55     TCNT1H = 0x08; // preload para 10 ms
56     TCNT1L = 0xF0;
57
58     if (running)
59     {
60         for (i = 0; i < 100; i++) {
61             incrementar_tiempo();
62         }
63     }
64 }

```

```

64 void main(void) {
65     // ===== PUERTOS =====
66     DDRA=0x00;
67     PORTA=0x00;
68
69     DDRB=0xFF;
70     PORTB=0x00;
71
72     DDRC=0x3F;
73     PORTC=0x00;
74
75     DDRD=0x00;
76     PORTD=0x07;
77
78     // ===== TIMER 0 =====
79     TCCR0=(0<<WGM00) | (0<<WGM01) | (0<<COM01) | (0<<COM00)
80           | (0<<CS02) | (1<<CS01) | (0<<CS00); // prescaler 8
81
82     TCNT0=131;
83
84     // ===== TIMER 1 =====
85     TCCR1A=0x00;
86     TCCR1B=(0<<WGM12) | (1<<CS11); // prescaler 8
87
88     TCNT1H=0x08;
89     TCNT1L=0xF0;
90
91     // ===== INTERRUPTOS =====
92     TIMSK=(1<<TOIE0) | (1<<TOIE1);
93
94     #asm("sei")

```

```
96     reset_tiempo();
97
98
99     while (1) {
100         // Place your code here
101         // ===== VARIABLES EST  TICAS PARA FLANCO =====
102         static unsigned char last_start = 1;
103         static unsigned char last_stop = 1;
104         static unsigned char last_reset = 1;
105
106         // ===== BOT  N START =====
107         if (!START_BUTTON && last_start) { // flanco de bajada
108             delay_ms(20);
109             if (!START_BUTTON) running = 1;
110         }
111         last_start = START_BUTTON;
112
113         // ===== BOT  N STOP =====
114         if (!STOP_BUTTON && last_stop) {
115             delay_ms(20);
116             if (!STOP_BUTTON) running = 0;
117         }
118         last_stop = STOP_BUTTON;
```

```
120         // ===== BOT  N RESET =====
121         if (!RESET_BUTTON && last_reset) {
122             delay_ms(20);
123             if (!RESET_BUTTON) {
124                 reset_tiempo();
125                 running = 0;
126             }
127         }
128         last_reset = RESET_BUTTON;
129     }
130 }
131
132 void actualizar_displays(void) {
133     PORTC = 0x00;
134
135     switch (display_actual) {
136     case 0:
137         PORTB = tabla_7_segmentos[minutos_decenas];
138         PORTC = 0x01;
139         break;
140     case 1:
141         PORTB = tabla_7_segmentos[minutos_unidades];
142         PORTC = 0x02;
143         break;
144     case 2:
145         PORTB = tabla_7_segmentos[segundos_decenas];
146         PORTC = 0x04;
147         break;
148     case 3:
149         PORTB = tabla_7_segmentos[segundos_unidades];
150         PORTC = 0x08;
151         break;
```



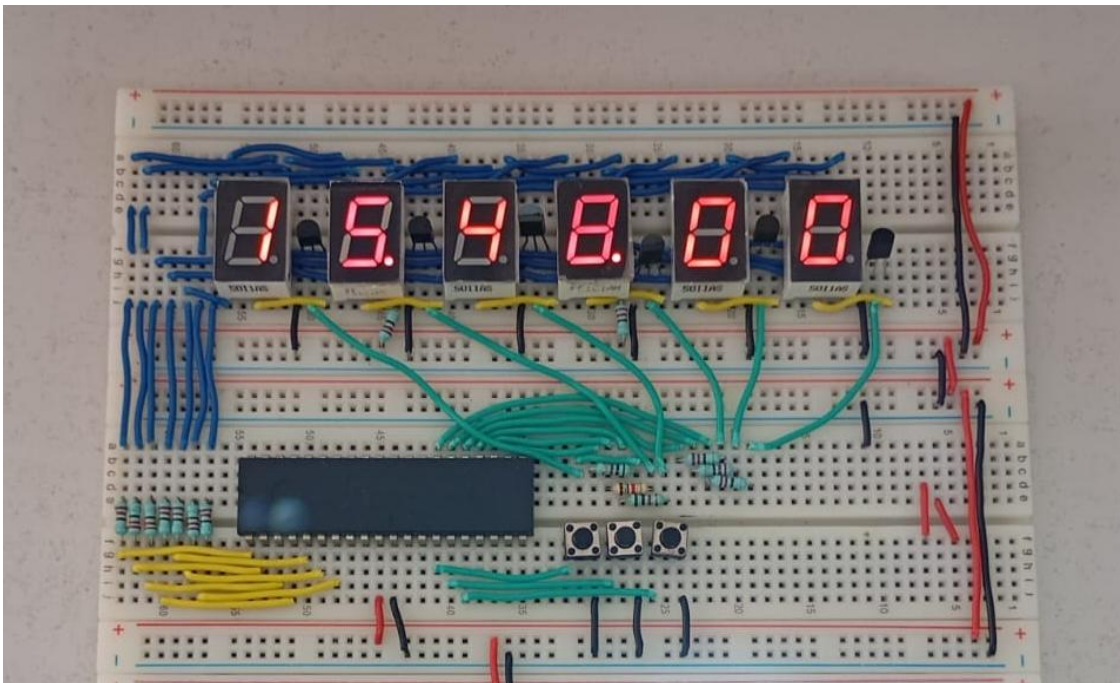
```
152     case 4:
153         PORTB = tabla_7_segmentos[centesimas_decenas];
154         PORTC = 0x10;
155         break;
156     case 5:
157         PORTB = tabla_7_segmentos[centesimas_unidades];
158         PORTC = 0x20;
159         break;
160 }
161
162 display_actual++;
163 if (display_actual > 5) display_actual = 0;
164 }
```

```
166 void incrementar_tiempo(void) {
167     centesimas_unidades++;
168     if (centesimas_unidades > 9) {
169         centesimas_unidades = 0;
170         centesimas_decenas++;
171         if (centesimas_decenas > 9) {
172             centesimas_decenas = 0;
173             segundos_unidades++;
174             if (segundos_unidades > 9) {
175                 segundos_unidades = 0;
176                 segundos_decenas++;
177                 if (segundos_decenas > 5) {
178                     segundos_decenas = 0;
179                     minutos_unidades++;
180                     if (minutos_unidades > 9) {
181                         minutos_unidades = 0;
182                         minutos_decenas++;
183                         if (minutos_decenas >= 10) {
184                             reset_tiempo();
185                         }
186                     }
187                 }
188             }
189         }
190     }
191 }
```



```
193 void reset_tiempo(void) {  
194     minutos_decenas = 0;  
195     minutos_unidades = 0;  
196     segundos_decenas = 0;  
197     segundos_unidades = 0;  
198     centesimas_decenas = 0;  
199     centesimas_unidades = 0;  
200  
201     display_actual = 0;  
202 }
```

Circuito físico



CONCLUSIÓN

- **García Escamilla Bryan Alexis**

En un inicio el código se hizo para que el cronómetro contara 100 minutos reales, sin embargo, a pesar de que esto es correcto, para comprobar el correcto funcionamiento del cronómetro esto resulta poco práctico, por lo que se optó por aumentar su velocidad 100 veces, de esta forma fue posible observar el conteo completo del cronómetro.

Para ello originalmente el Timer1 interrumpía cada 10 ms y llamaba a la función **incrementar_tiempo** la cual le sumaba al cronómetro 1 centésima (0.01s). Para aumentar el tiempo sumado al cronómetro lo que se hizo fue colocar la función **incrementar_tiempo** dentro de un bucle que se repitiera 100 veces, para que ahora por

los cada 10 ms que se hace la interrupción se le suma al cronómetro 100 centésimas (1 segundo), haciendo que este ahora avance 1 segundo cada 10 ms.

Con la velocidad aumentada 100 veces ahora el cronómetro ya no tarda 100 minutos reales sino ahora 1 minuto. Sabemos esto partiendo que el tiempo máximo del cronómetro es 99:59:99 que equivalen a 6000 segundos y tomando la siguiente relación de tiempo

$$Tiempo\ real = \frac{6000\ s}{N}$$

y sabiendo que el bucle se repite $N = 100$ veces entonces

$$Tiempo\ real = \frac{6000\ s}{100} = 60\ s = 1\ min$$

- **Meléndez Macedonio Rodrigo**

Para esta práctica se desarrolló un cronómetro similar al de la práctica 8, sin embargo, para esta ocasión en lugar de trabajar con un cronómetro de 60 segundos, se trabajó con uno de 100 minutos ml que implica un poco más de trabajo, ya que para este tuvimos que utilizar 6 displays para lograr el objetivo principal. Los primeros dos displays corresponden a los minutos (0-99), los siguientes dos corresponden a los segundos (0-59) y el último par corresponde a los milisegundos. (0-99).

Al tratarse de 100 minutos, evidentemente no nos podemos esperar dicha cantidad de tiempo para corroborar su funcionamiento, por lo tanto, decidimos modificar la función **incrementar_tiempo** de tal manera que se repitiera dentro de un bucle para aumentar la velocidad del conteo 100 veces y en su lugar, cronometrar los 100 minutos en 1 minuto real.

BIBLIOGRAFÍA

- (s.f.). *Bidirectional Counters*. Electronic Tutorials. Recuperado el 13 de abril de 2026, de https://www.electronics-tutorials.ws/counter/count_4.html
- (s.f.). *Multiplexing Displays*. All About Circuits. Recuperado el 13 de abril de 2026, de <https://www.allaboutcircuits.com/>
- Proteus Design Suite – *Documentación oficial sobre simulación de multiplexación y manejo de displays*.